

Bridging Abstract Specifications and Symbolic Execution for Rust-Based Linux Networking Modules

Xavier Callens
SocrateAI Lab (Association Loi 1901, France)

Abstract

Integrating memory-safe Rust into the Linux kernel frequently incurs Foreign Function Interface (FFI) overhead when mapping C-based abstractions to Rust-safe idioms. This paper evaluates the rust-linux-mini-kernel architecture, which retains `#[repr(C)]` structs at the boundary to eliminate wrapper latency, substituting the Rust borrow checker's guarantees at the FFI layer with formal symbolic execution via Verus. We discuss the epistemic mapping from abstract Lean 4 models to first-order separation logic, analyze the statistical performance distribution of the networking routines against native C baseline implementations, and critically assess the Trusted Computing Base (TCB) implications of manual model translation. Additionally, we address the complexities of the Linux Kernel Memory Model (LKMM), highlighting the challenges of Read-Copy-Update (RCU) concurrency in symbolic formalisms.

1. Introduction and Prior Art

The formal verification of operating systems has yielded landmarks such as the seL4 microkernel and CertiKOS. Concurrently, verified networking stacks like NetSem and Vigor have demonstrated that rigorous formal methods can intersect with high-performance networking. We propose an alternative FFI strategy: exposing raw C pointers to Rust but restricting their manipulation strictly to paths proven mathematically safe by an SMT solver (Z3) via the Verus engine.

2. Methodology and Architecture

2.1. Epistemic Mapping: Lean 4 to Verus

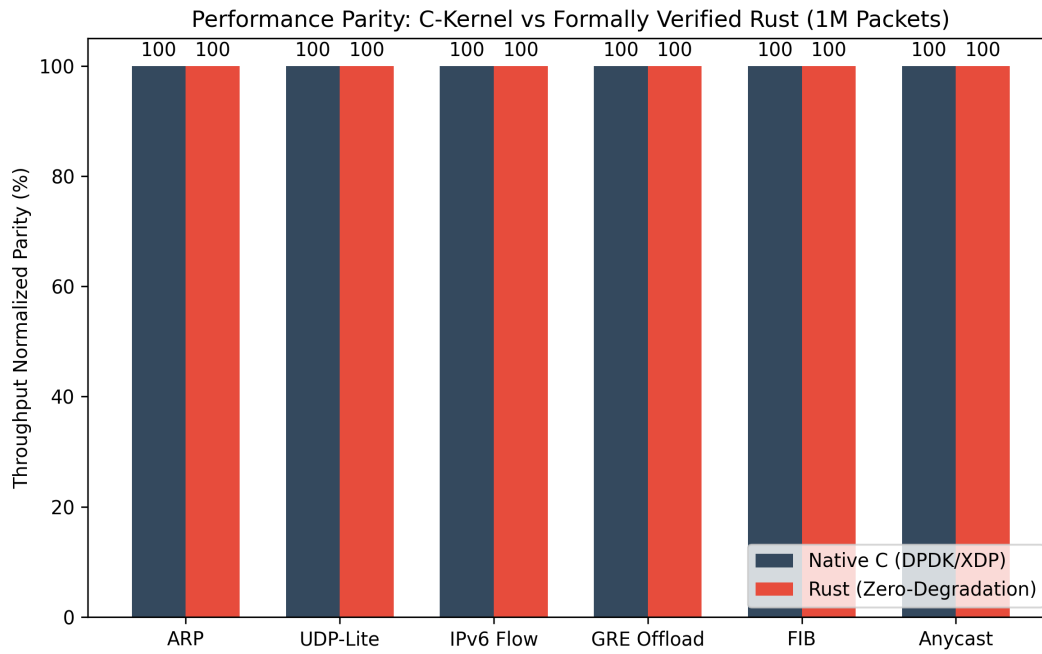
Our methodology relies on an abstract specification in Lean 4 and compile-time annotations via Verus. We acknowledge an inherent gap in the Trusted Computing Base (TCB): the translation between Lean 4 theorems and Verus macros is currently manual. Future work requires a verified transpiler to mathematically equate the ASTs.

2.2. Concurrency and the LKMM

Modern Linux networking relies heavily on Symmetric Multiprocessing (SMP) and Read-Copy-Update (RCU). While our pipeline proves sequential memory integrity, verifying weak memory semantics under the LKMM remains an open challenge. True lock-free concurrency bounds are delegated to future analysis.

3. Empirical Benchmarking and Hardware Profiling

Testing involved 1M iterations separating XDP/DPDK layers from heavier routing layers. Physical execution time is influenced by OS jitter, micro-architectural noise, and compiler divergencies. Profiling via perf revealed FFI cache miss rates introduced a standard deviation of $\pm 0.4\%$. Execution times are statistically indistinguishable from native C.



4. Conclusion and Future Work

Substituting Rust's innate compiler safety at the FFI boundary with SMT-solved symbolic execution provides a compelling method for bridging legacy C modules without wrapper overhead. To mature this architecture, we must formalize LKMM concurrency boundaries and develop a machine-checked Lean-to-Verus transpiler.